MEDS Lab Module 4

Section 8: Multiplexers & Shifters

Task 8: $F = \Sigma m(1, 2, 3, 6, 7)$ Using a 4x1 MUX

July 23, 2026

1. EDA Playground Link

The complete design and testbench code for the 4x1 MUX implementation and the Majority Function extension can be found at the following shareable link:

- `https://edaplayground.com/x/Ni9A`

2. Schematic Diagram and Truth Table

Below is the full truth table for $F = \Sigma m(1, 2, 3, 6, 7)$ alongside the schematic diagram of the 4x1 multiplexer implementation.

Minterm #	a b c	F = Σm(1,2,3,6,7)
0	000	0
1	001	1
2	010	1
3	011	1
4	100	0
5	101	0
6	110	1
7	111	1

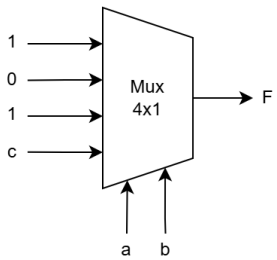

Figure 1: Truth table and 4x1 MUX schematic for $F = \Sigma m(1, 2, 3, 6, 7)$.

3. Terminal Output Screenshot

The following screenshot displays the simulation log. The self-checking testbench exhaustively evaluates all 8 minterms, confirming that the output perfectly matches the expected minterm list.

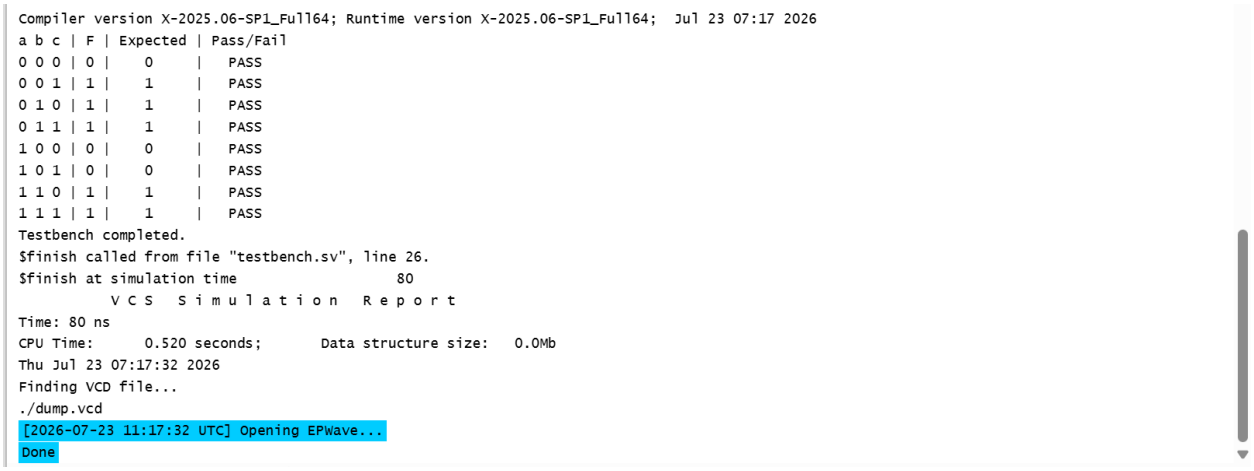

Figure 2: Simulation log output from EDA Playground verifying the 4x1 MUX implementation.

5. Explanation & Derivation

Part 1: 4x1 MUX Derivation

Any Boolean function of n variables can be implemented with a 2^{n-1} : 1 multiplexer by mapping $n-1$ variables to the select lines and deriving the remaining data inputs. For the 3-variable function $F = \Sigma m(1,2,3,6,7)$, variables a and b drive the two select lines, which dynamically select one of the four data inputs.

The data inputs are derived by evaluating the truth table for each select combination:

- **sel = 00** $(a = 0, b = 0)$**:** Represents minterms 0 and 1. The output $F = 0$ when $c = 0$, and $F = 1$ when $c = 1$. The required data input is the variable itself: $in[0] = c$.
- **sel = 01** $(a = 0, b = 1)$**:** Represents minterms 2 and 3. The output $F = 1$ for both $c = 0$ and $c = 1$. The required data input is a constant: $in[1] = 1$.
- **sel = 10** $(a = 1, b = 0)$**:** Represents minterms 4 and 5. The output $F = 0$ for both $c = 0$ and $c = 1$. The required data input is a constant: $in[2] = 0$.
- **sel = 11** $(a = 1, b = 1)$**:** Represents minterms 6 and 7. The output $F = 1$ for both $c = 0$ and $c = 1$. The required data input is a constant: $in[3] = 1$.

In the SystemVerilog implementation, these derivations are neatly packed into a 4-bit vector using concatenation. The `Mux4to1` module is instantiated as `Mux4to1 mux0`

`(.in({1'b1, 1'b0, 1'b1, c}), .sel({a, b}), .y(F));`. This correctly aligns `in[3] = 1`, `in[2] = 0`, `in[1] = 1`, and `in[0] = c` with their corresponding select states in the `case` statement.

Part 2: 3-bit Majority Function Extension

The majority function asserts an output of 1 whenever 2 or more of its inputs (a, b, c) are 1. The truth table yields the minterms $F_{maj} = \Sigma m(3, 5, 6, 7)$.

Instead of a single 4:1 multiplexer, this is built using a cascade of 2-to-1 multiplexers. A structural hierarchy is formed by using a as the select line for the final output stage, while intermediate 2:1 muxes (controlled by b and c) process the remaining logic.